

Desarrollo Web para portal de Inteligencia Analítica

Lección 3. Controles de Streamlit y consumo de APIs externas

Introducción

Hoy en día, los datos son uno de los activos más valiosos para cualquier organización. Clientes, proveedores y usuarios necesitan entender qué pasa con su información de manera rápida y sencilla. Por eso, es imprescindible contar con herramientas que faciliten su interpretación y permitan tomar decisiones rápidas, sencillas y efectivas.

Aunque existen versátiles marcos de trabajo de **Python** para el desarrollo web, como **Flask** y **Django**, la mayoría de los científicos de datos o ingenieros de aprendizaje automático no buscan convertirse en desarrolladores web. Necesitan una herramienta **fácil de aprender, rápida, sencilla y potente** para mostrar datos y recopilar los parámetros necesarios para el modelado.

Si bien, existen muchas herramientas, **Streamlit** ha llegado para hacer esta tarea mucho más fácil y rápida. Es un *framework* ligero, intuitivo y pensado para quienes manejan **Python**, y funciona como un marco web que permite crear aplicaciones web interactivas basadas en datos.



Naturalmente, aprender a usarlo requiere pruebas y errores. Por eso, en esta lección, estudiaremos la estructura de Streamlit, las formas básicas para construir una aplicación web y cómo crear una app interactiva en ciencia de datos.

Controles básicos para crear *dashboards* para la interacción con el usuario

Las aplicaciones web deben mostrar datos y cifras relacionadas según la entrada del usuario, en ese sentido, es una buena idea colocar controles de usuario interactivos, conocidos como *widgets*. Existen muchos, pero comenzaremos con uno de los más relevantes: **la barra lateral (*sidebar*)**. Esta separación deja en claro la intención de la interactividad para los usuarios. Su función es proporcionar parámetros (la entrada), mientras que la pantalla principal es para proporcionar comentarios (la salida), en respuesta a la entrada de los usuarios. Para tener una idea de esto, observa la siguiente figura.



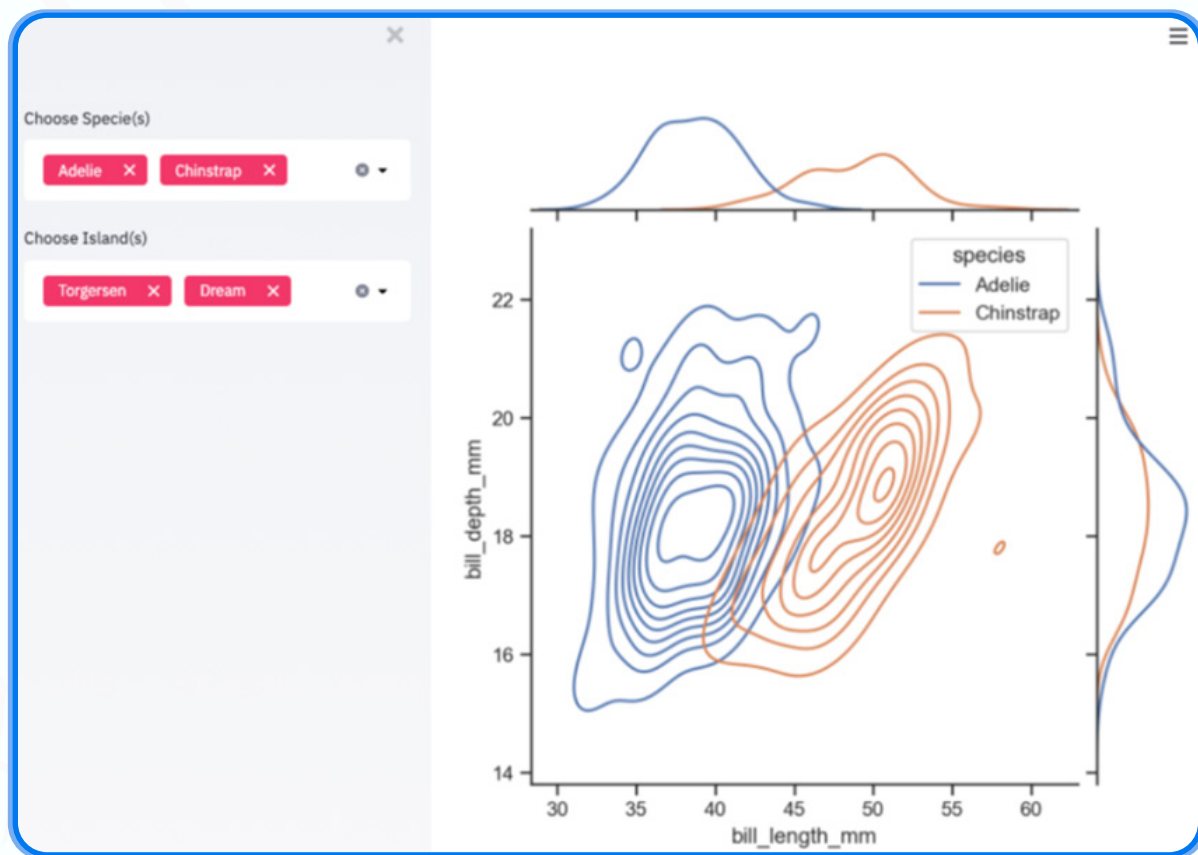


Figura 1. Uso de la barra lateral (*sidebar*).

Ahora, si tiene más parámetros, el uso de la barra lateral (*sidebar*) puede mejorar la limpieza de la aplicación. Sin embargo, debes considerar que el ancho de esta barra es limitado, así que debes evitar los textos largos y los *widgets* grandes, ya que pueden hacer que el flujo de información se sienta roto. Además, si existen demasiados parámetros que exceden la altura de la pantalla, es un buen momento para pensar en reorganizar los *widgets*.

Con esto en mente, ha llegado el momento de construir una aplicación en Python que contenga una barra lateral que sirva para alojar los datos de entrada del proyecto de ciencia de datos, así mismo, dicho archivo contendrá la información que se desplegará en la página principal. Vamos a editar el código en Visual Studio Code y guardarlo con el nombre: **sidebarApp.py**.

Para hacerlo, debes realizar lo siguiente:

Paso 1. Activa el ambiente virtual creado en el tema anterior, ¿lo recuerdas?

```
$ source streamlit-env/bin/activate
```

```
$ touch sidebarApp.py
```

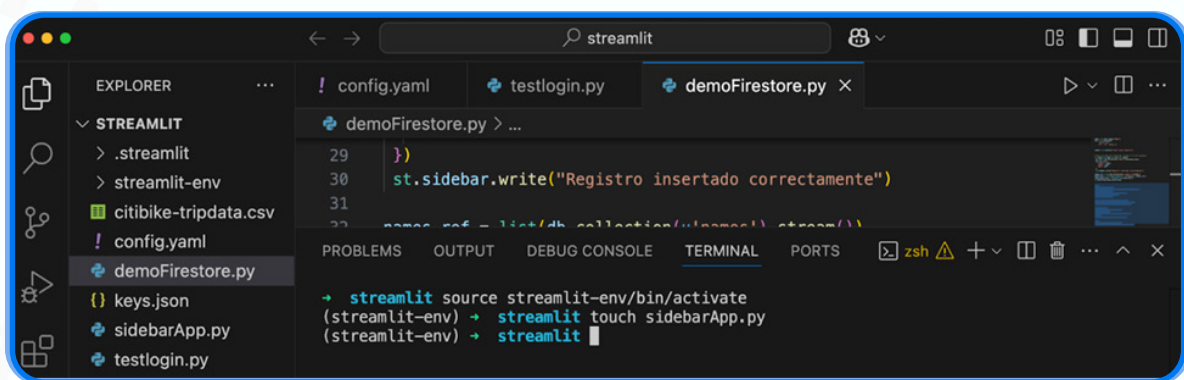


Figura 2. Ambiente virtual

El código para editar **sidebarApp.py** es:

```
import streamlit as st

# Crear el título para la aplicación web
st.title("Mi Primera App con Streamlit")

sidebar = st.sidebar

sidebar.title("Esta es la barra lateral.")

sidebar.write("Aquí van los elementos de entrada.")
```

```
st.header("Información sobre el Conjunto de Datos")

st.header("Descripción de los datos")

st.write("""

Este es un simple ejemplo de una app para predecir

;Esta app predice mis datos!

""")
```

La primera instrucción del archivo es la importación de la librería **Streamlit**:

```
import streamlit as st
```

La segunda instrucción es la creación de un título para la aplicación:

```
st.title("Mi Primera App con Streamlit")
```

Paso 2. Crea una sección del lado derecho, denominada **barra lateral o sidebar**. Para esto, utiliza la siguiente instrucción:

```
sidebar = st.sidebar
```

Paso 3. Una vez creada la barra lateral, agrega un título y un párrafo que explique lo que vas a encontrar en dicha sección.

Las instrucciones para hacerlo son:

```
sidebar.title("Esta es la barra lateral.")
```

```
sidebar.write("Aquí van los elementos de entrada.")
```

Paso 4. Ahora, genera subtítulos para dar más contexto sobre la aplicación, para esto, utiliza la instrucción: `st.header`. A continuación, observa las dos instrucciones:

```
st.header("Información sobre el Conjunto de Datos")
```

```
st.header("Descripción de los datos ")
```

Paso 5. Agrega un pequeño párrafo que muestre una breve descripción de la aplicación web. Para hacerlo, utiliza una instrucción que revisaste en el tema anterior: `st.write`. A continuación, observa dicha instrucción:

```
st.write("""
```

```
Este es un simple ejemplo de una app para predecir
```

```
¡Esta app predice mis datos!
```

```
""")
```

Paso 6. Ejecuta la aplicación con:

```
$ streamlit run sidebarApp.py
```

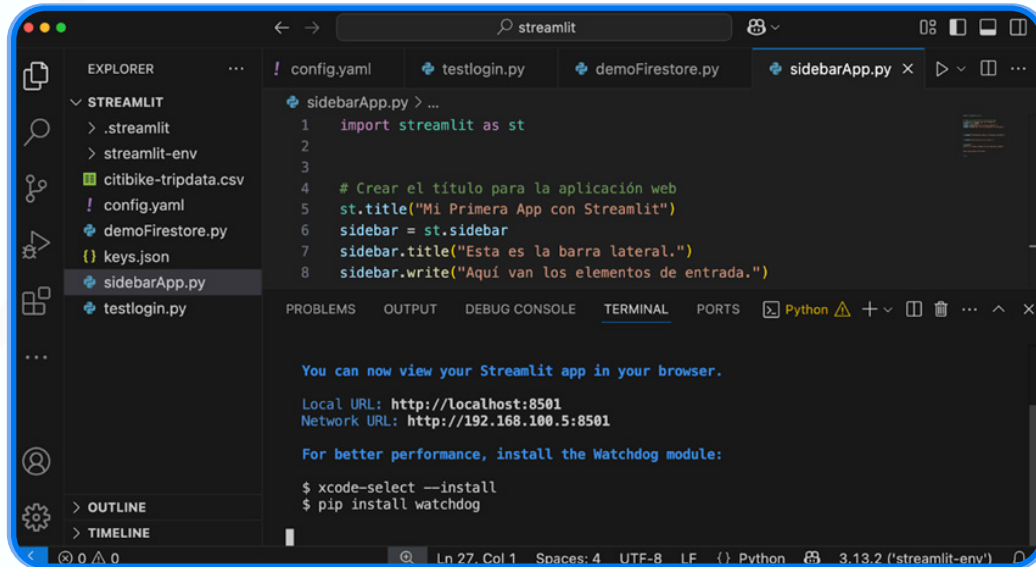


Figura 3. Ejecución de la **sidebarApp.py**.

Paso 7. Al hacerlo, se cargará una pestaña en el navegador predeterminado. Visualiza la aplicación web que has creado.

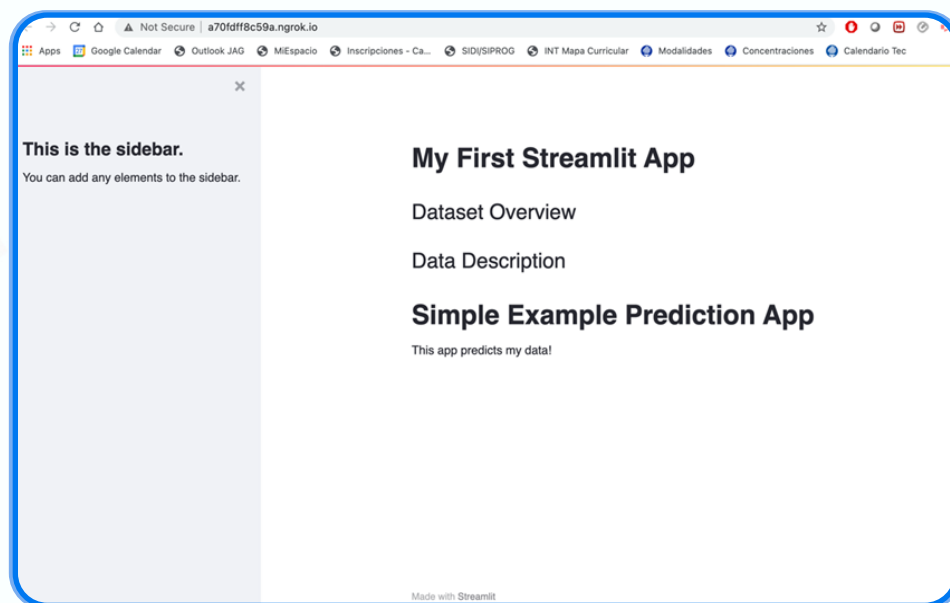


Figura 4. Vista de la aplicación web con Streamlit usando una barra lateral (*sidebar*).

Estos son los conocimientos que necesitas para generar una barra lateral para una aplicación web utilizando el marco de referencia de **Streamlit**, pero ¿cómo construir un sitio web que contenga los siguientes elementos?

- Una sección de bienvenida que contenga una barra lateral.
- Una sección que contenga un apartado para tener *checkboxes*.
- Una sección que contenga un apartado para tener *selectboxes*.

En el siguiente enlace encontrarás las instrucciones necesarias para contestar estas interrogantes: https://docs.streamlit.io/en/stable/getting_started.html. Cópialo y pégalo en tu navegador para acceder al contenido.

Nota: Los enlaces a las páginas *web* incluidos en esta bibliografía fueron consultados y verificados por última vez el 13 de mayo de 2025 y su disponibilidad depende del autor.

Por otra parte, a partir de los **widgets**, **Streamlit** permite crear interactividad directamente en las aplicaciones como **casillas de verificación, controles de fecha, botones, controles deslizantes y más**. Estas posibilidades son muy significativas para los científicos de datos, porque les permiten crear elementos para realizar análisis más profundos. Existen muchos **widgets**, pero nos vamos a centrar en los 4 más importantes: **controles deslizantes o sliders; botones de opciones o radio buttons, controles checkbox y entradas de fecha** `date_input`.

Además, para recopilar las respuestas de los usuarios, en este ejemplo, recopila la fecha actual con el control: `st.date_input()`, la fecha actual con la función: `datetime.date.today()`, y para sugerirla como fecha por *default* al control, utiliza: `st.date_input()`. En caso de que la fecha haya sido seleccionada de forma adecuada, usa `st.success()` para enviar un mensaje con la fecha seleccionada. Para hacerlo, sigue con los pasos:

Paso 8. En **Visual Studio Code**, crea:

```
$ touch dateApp.py
```

Paso 9. Edita **dateApp.py** con el siguiente código:

```
import streamlit as st

import datetime

# Give user the current date

today = datetime.date.today()

today_date = st.date_input('Current date', today)

st.success('Current date: `%s`' % (today_date))
```

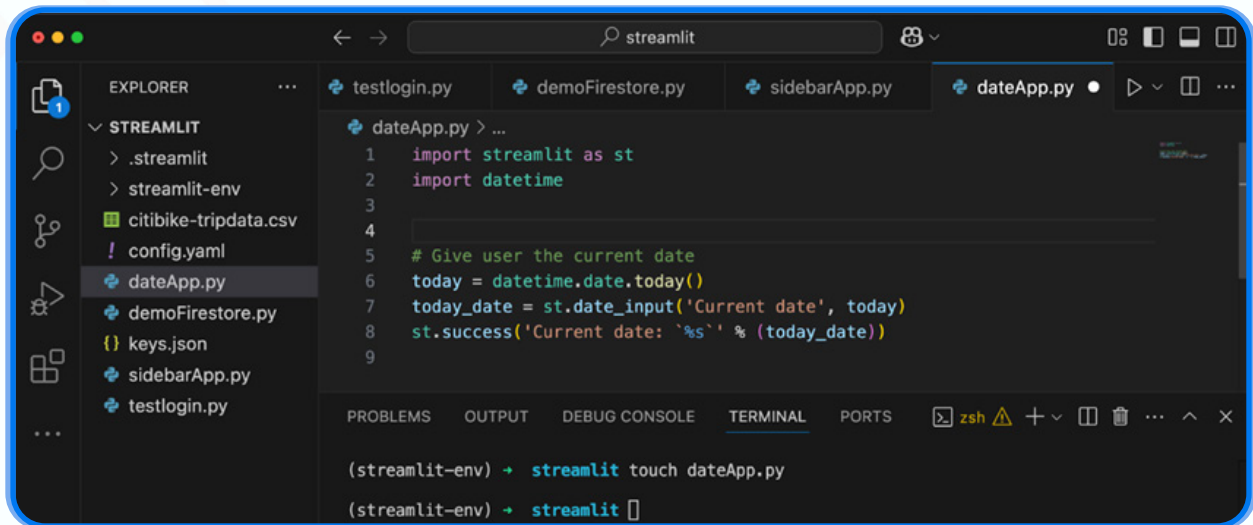


Figura 5. Ejecución del código.

Paso 10. Ejecuta la aplicación con:

```
$ streamlit run dateApp.py
```

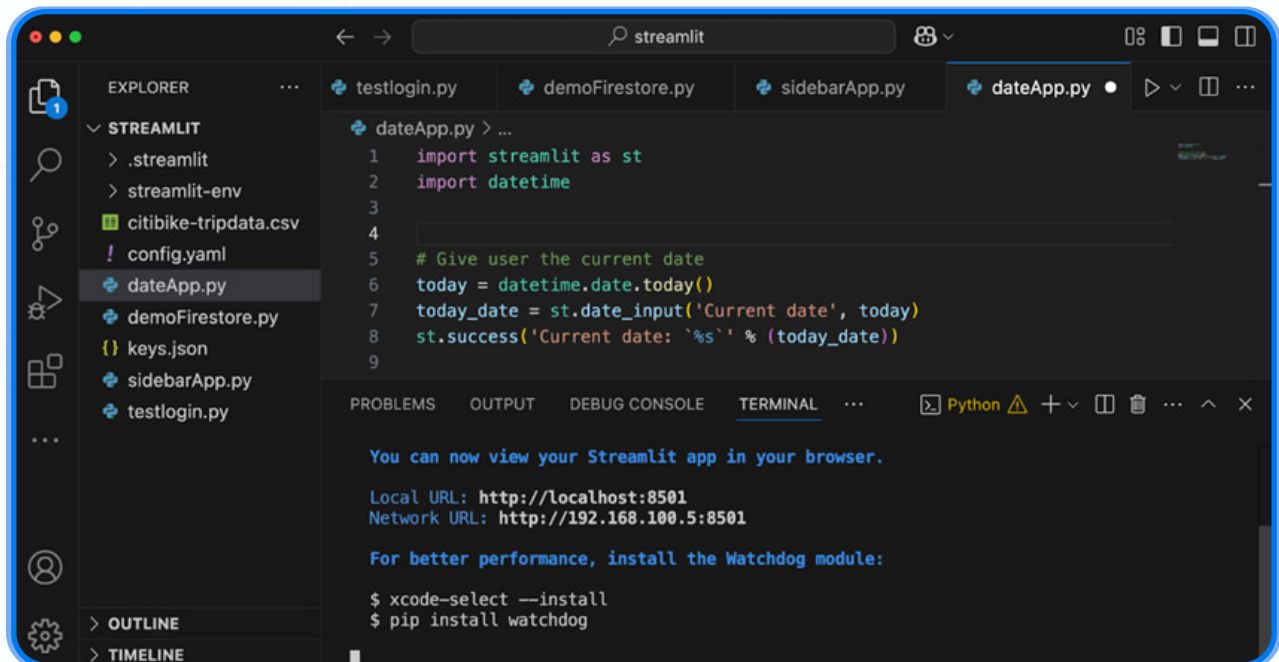


Figura 6. Ejecución de **dateApp.py**.

Paso 11. Observa la figura que muestra el componente `st.date_input` generado con el código anterior.

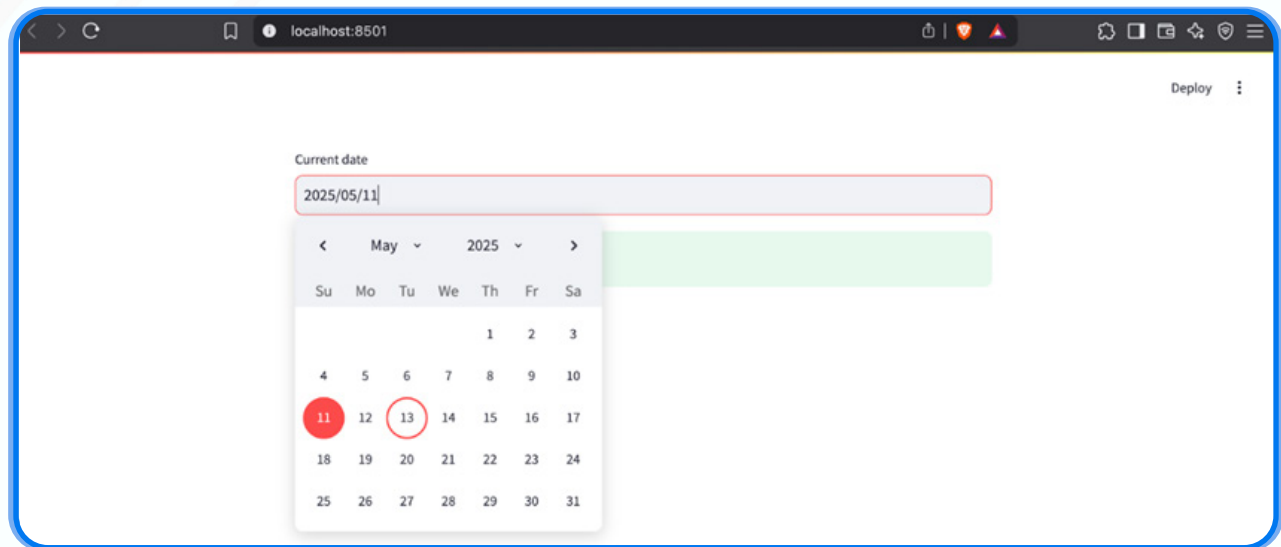


Figura 7. Aplicación **Streamlit** con función `st.date_input`.

Por otra parte, la siguiente sección utiliza el control *checkbox* para preguntar al usuario si deseamos o no mostrar una tabla de datos. Una vez que se obtiene la respuesta, se evalúa usando un `if`, si el resultado es *true*, entonces, se imprime el *dataframe*. Para esto, continúa con los pasos:

Paso 12. En **Visual Studio Code**, crea:

```
$ touch checkboxApp.py
```

Paso 13. Edita **checkboxApp.py** con el siguiente código:

```
import streamlit as st
```

```
import pandas as pd
```

```

titanic_data = pd.read_csv('https://raw.
githubusercontent.com/mwaskom/seaborn-data/master/
titanic.csv'

)

# Display the content of the dataset if checkbox is
true

st.header("Dataset")

agree = st.checkbox("show DataSet Overview ?")

if agree:

    st.dataframe(titanic_data)

```

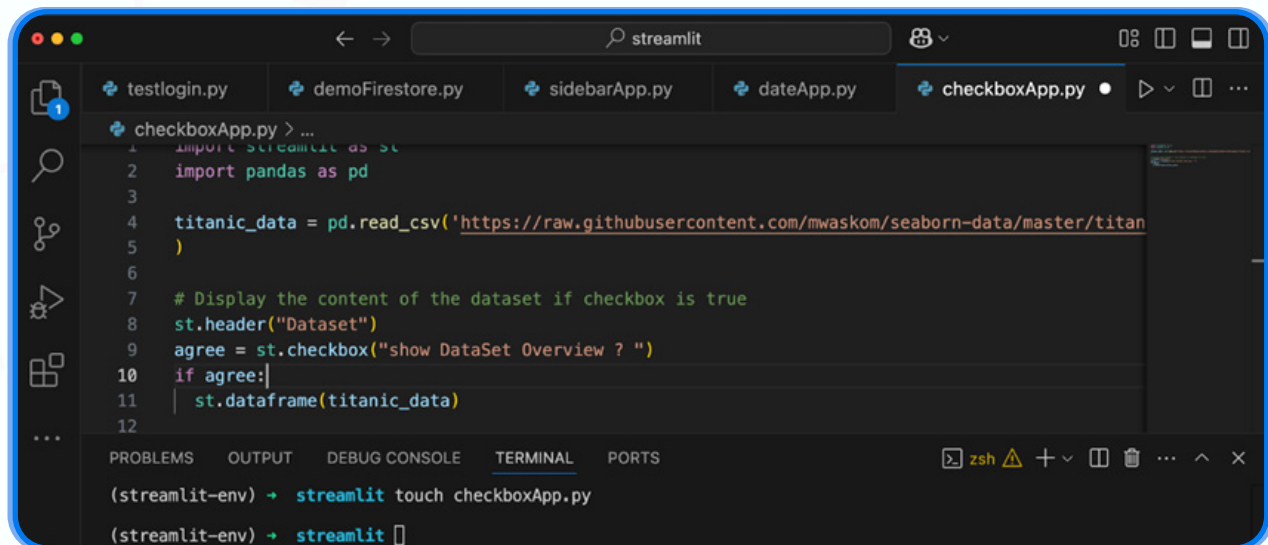
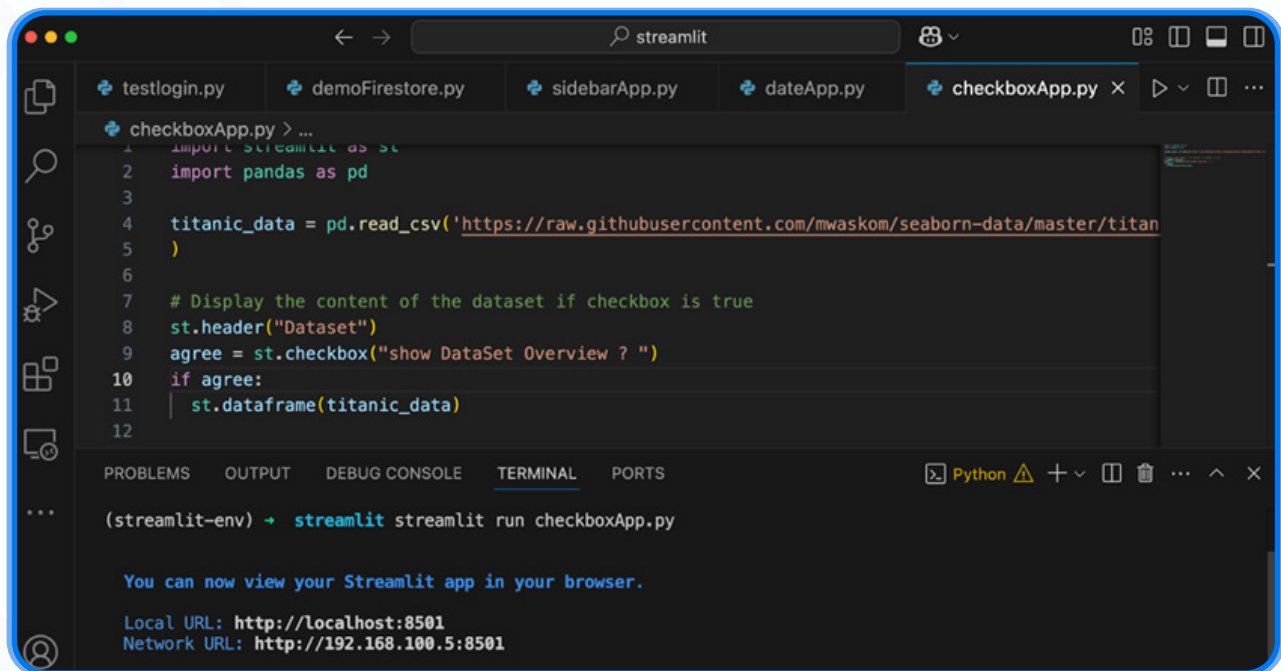


Figura 8. Edición de **checkboxApp.py**.

\$ streamlit run checkboxApp.py



```
checkboxApp.py > ...
1 import streamlit as st
2 import pandas as pd
3
4 titanic_data = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv')
5
6
7 # Display the content of the dataset if checkbox is true
8 st.header("Dataset")
9 agree = st.checkbox("show DataSet Overview ? ")
10 if agree:
11     st.dataframe(titanic_data)
12
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

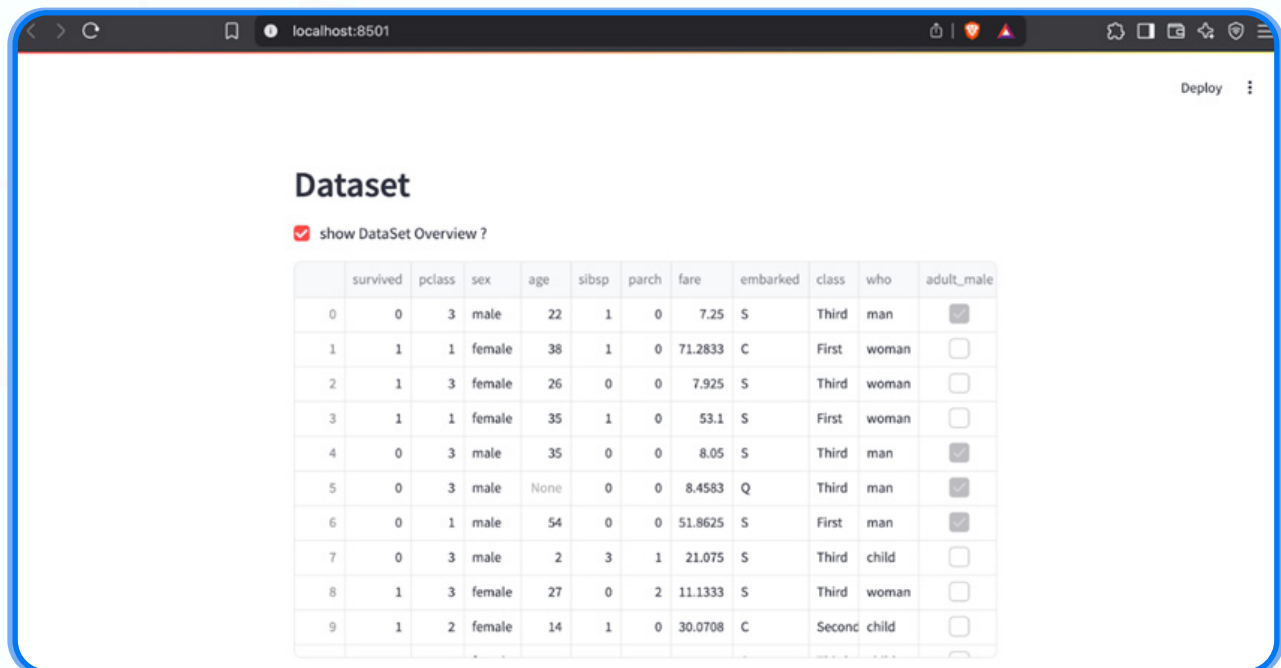
(streamlit-env) → streamlit run checkboxApp.py

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>
Network URL: <http://192.168.100.5:8501>

Figura 9. Edición de **checkboxApp.py**.

Paso 14. Observa la figura que muestra el uso de *checkbox* en **Streamlit**:



	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male
0	0	3	male	22	1	0	7.25	S	Third	man	☒
1	1	1	female	38	1	0	71.2833	C	First	woman	☐
2	1	3	female	26	0	0	7.925	S	Third	woman	☐
3	1	1	female	35	1	0	53.1	S	First	woman	☐
4	0	3	male	35	0	0	8.05	S	Third	man	☒
5	0	3	male	None	0	0	8.4583	Q	Third	man	☒
6	0	1	male	54	0	0	51.8625	S	First	man	☒
7	0	3	male	2	3	1	21.075	S	Third	child	☐
8	1	3	female	27	0	2	11.1333	S	Third	woman	☐
9	1	2	female	14	1	0	30.0708	C	Second	child	☐

Figura 10. Selección de opciones con el botón de opción (*radio button*).

Para recopilar las respuestas de los usuarios, tienes varias opciones para crear *widgets* de opción múltiple. El primer *widget* es el botón de opción (*radio button*). La instrucción `st.radio` se debe aplicar a una variable que para este ejemplo se denominará `selected_class`. La instrucción tiene dos secciones: una indica el título del botón de opción, y la otra, indica la información que alimentará dicho botón. Para este ejemplo, es la columna **clase** del conjunto de datos del **Titanic**. Adicionalmente, debes utilizar la instrucción `st.write` para plasmar el botón de opción en tu aplicación, y, además, capturar la opción que el usuario haya seleccionado. Para hacerlo, continúa:

Paso 15. En **Visual Studio Code**, crea:

```
$ touch radioApp.py
```

Paso 16. Edita **radioApp.py** con el siguiente código:

```
import streamlit as st

import pandas as pd

titanic_data = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv')

selected_class = st.radio("Select Class", titanic_data['class'].unique())

st.write("Selected Class:", selected_class)
```



The screenshot shows a web browser window with a single tab titled 'radioApp.py'. The address bar shows 'streamlit'. The browser's developer tools are open, displaying the code for 'radioApp.py' in a dark-themed editor. The code is as follows:

```
1 import streamlit as st
2 import pandas as pd
3
4 titanic_data = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv')
5
6 selected_class = st.radio("Select Class", titanic_data['class'].unique())
7 st.write("Selected Class:", selected_class)
8
9
10
11
```

Below the code editor, the 'TERMINAL' tab is active, showing the following output:

```
(streamlit-env) → streamlit run radioApp.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.100.5:8501
```

Figura 12. Edición de **radioApp.py**.

Paso 17. Al hacerlo, observa algo parecido a lo que se muestra a continuación. Considera que la opción seleccionada será el valor real de las opciones mostradas en lugar del índice numérico seleccionado. En este caso, será el texto:

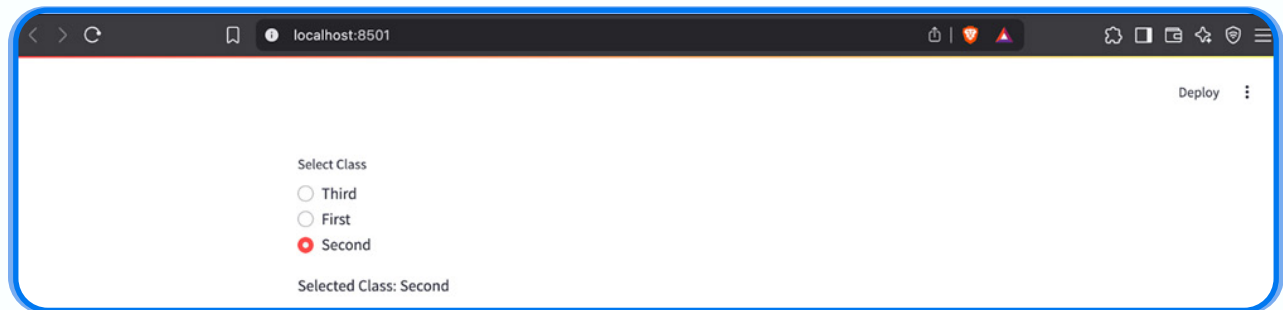


Figura 13. Selección de opciones con el botón de opción (*radio button*).

El *widget* `st.selectbox` proporciona una lista desplegable de datos que le permite al usuario elegir entre una lista de opciones. Para esto, continúa:

Paso 18. Crea el código en **Visual Studio Code**:

```
$ touch selectApp.py
```

Paso 19. Edita **selectApp.py** con el código:

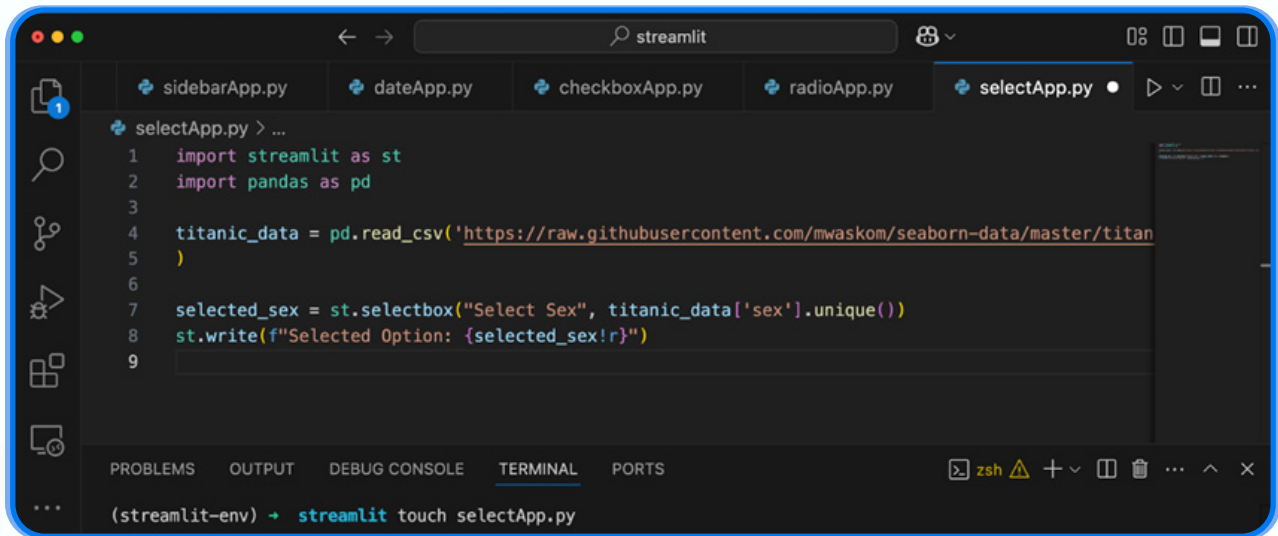
```
import streamlit as st

import pandas as pd

titanic_data = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv')
)
```

```
selected_sex = st.selectbox("Select Sex", titanic_data['sex'].unique())
```

```
st.write(f"Selected Option: {selected_sex!r}")
```



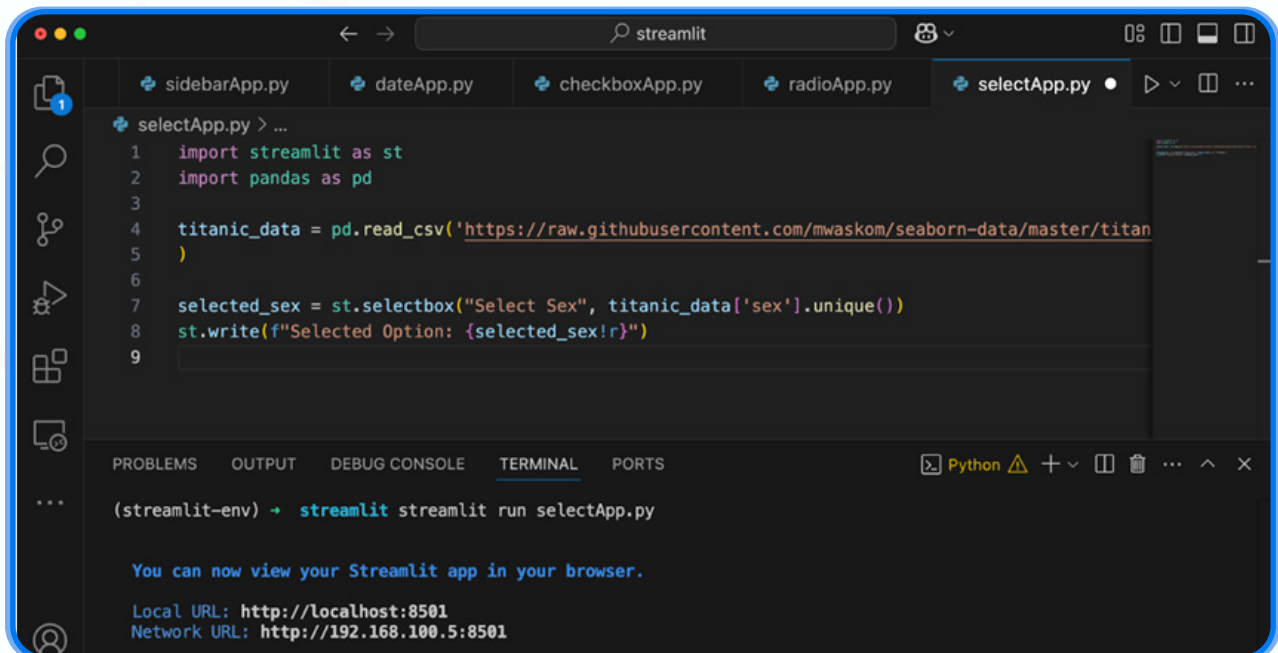
```
selectApp.py > ...
1 import streamlit as st
2 import pandas as pd
3
4 titanic_data = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv')
5
6
7 selected_sex = st.selectbox("Select Sex", titanic_data['sex'].unique())
8 st.write(f"Selected Option: {selected_sex!r}")
9
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(streamlit-env) → streamlit touch selectApp.py

Figura 14. Edición de **selectApp.py**.

```
$ streamlit run selectApp.py
```



```
selectApp.py > ...
1 import streamlit as st
2 import pandas as pd
3
4 titanic_data = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv')
5
6
7 selected_sex = st.selectbox("Select Sex", titanic_data['sex'].unique())
8 st.write(f"Selected Option: {selected_sex!r}")
9
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(streamlit-env) → streamlit run selectApp.py

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>
Network URL: <http://192.168.100.5:8501>

Figura 15. Edición de **selectApp.py**.

La casilla de verificación es mucho más sencilla, pues solo requiere dos instrucciones. La primera es la asignación a una variable, que para este ejercicio se denomina `selected_sex`; a la instrucción `st.selectbox`. La instrucción `st.selectbox` tiene dos parámetros: el nombre de la casilla de verificación y el conjunto de datos de donde se utilizará para generar dicha casilla, en este caso, utilizamos la columna denominada **sex** del conjunto de datos del **Titanic**. Posteriormente, la segunda instrucción, `st.write`, permite desplegar la casilla de verificación.

Nota importante: En la instrucción `st.write` debes notar que para el título se antepone la letra “f”, esta indica que se capturará información, mientras que en la sección donde se indica la variable, al final se pone el signo de admiración y la letra “r”, esto indica que se escribirá la opción que se haya elegido.

Entonces, la interacción se verá de la siguiente manera:

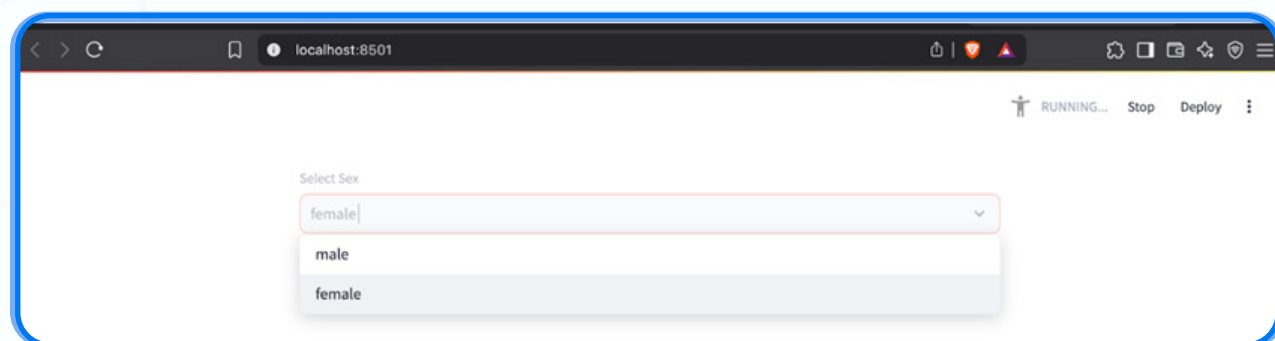


Figura 16. Selección de opciones con la casilla de verificación (*selectbox*).

Además de este *widget*, **Streamlit** también proporciona un control deslizante (`st.slider`), que permite al usuario especificar una entrada de número sin teclear y solo seleccionado el valor deseado en una barra deslizante. Para hacerlo continúa con los pasos:

Paso 20. En **Visual Studio Code**, crea:

```
$ touch slidertApp.py
```

Paso 21. Edita **sliderApp.py** con el siguiente código:

```
import streamlit as st

import pandas as pd

titanic_data = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv')

)

optionals = st.beta_expander("Optional
Configurations", True)

fare_select = optionals.slider(

    "Select the Fare",

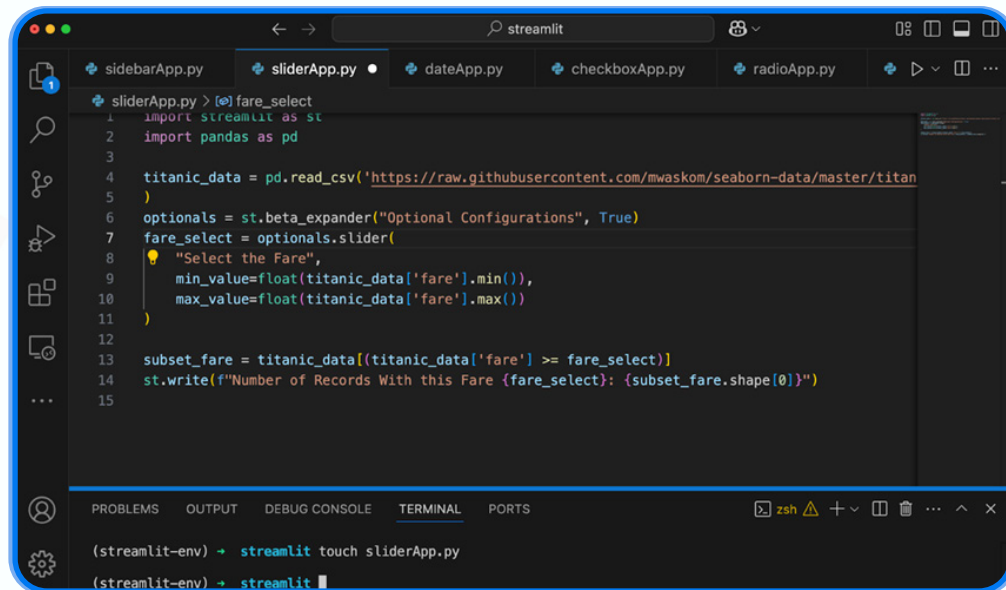
    min_value=float(titanic_data['fare'].min()),

    max_value=float(titanic_data['fare'].max())

)
```

```
subset_fare = titanic_data[(titanic_data['fare'] >= fare_select)]
```

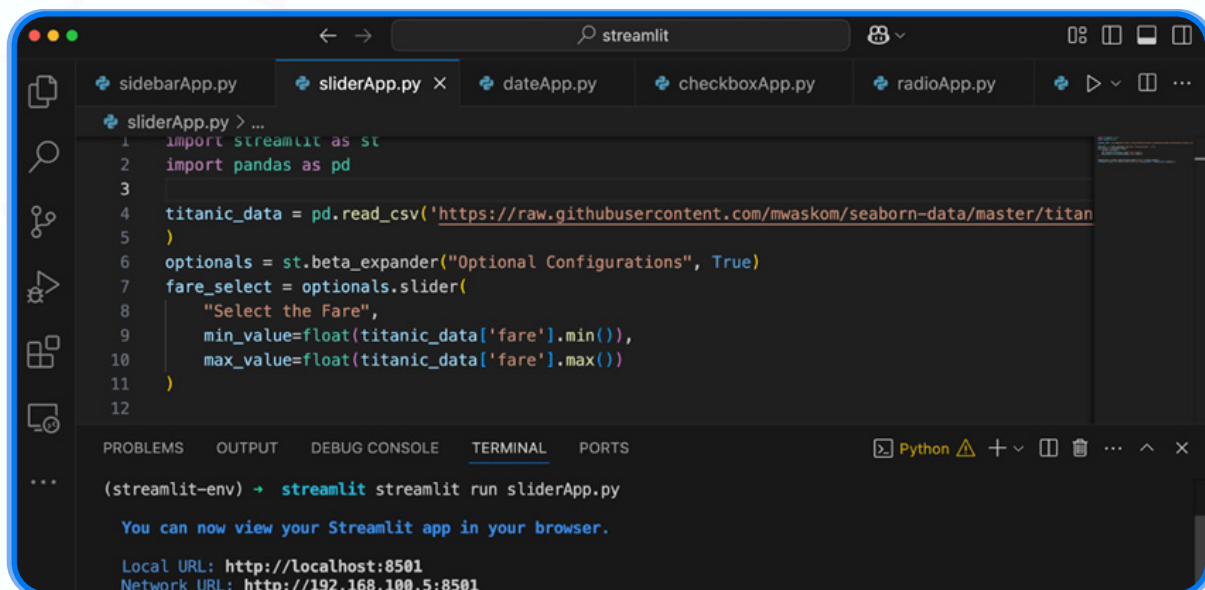
```
st.write(f"Number of Records With this Fare {fare_select}: {subset_fare.shape[0]}")
```



```
1 import streamlit as st
2 import pandas as pd
3
4 titanic_data = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv')
5
6 optional = st.beta_expander("Optional Configurations", True)
7 fare_select = optional.slider(
8     "Select the Fare",
9     min_value=float(titanic_data['fare'].min()),
10    max_value=float(titanic_data['fare'].max())
11 )
12
13 subset_fare = titanic_data[(titanic_data['fare'] >= fare_select)]
14 st.write(f"Number of Records With this Fare {fare_select}: {subset_fare.shape[0]}")
15
```

Figura 17. Edición de **sliderApp.py**.

```
$ streamlit run slidertApp.py
```



```
(streamlit-env) → streamlit run sliderApp.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.100.5:8501
```

Figura 18. Edición de **sliderApp.py**.

Paso 22. Observa que la primera instrucción establece una variable que contendrá el expansor, con el parámetro `True` para que se expanda automáticamente. Posteriormente, se establece una variable que contendrá el valor seleccionado en la barra deslizante, para este ejemplo, utilizamos las tarifas del conjunto de datos del **Titanic**. Esta variable utiliza la instrucción `nombre_expansor.slider` para generar los máximos y mínimos. Cabe mencionar que esta instrucción utiliza datos que son numéricos, como precios, costos, temperatura, entre otros.

Paso 23. En este punto, y para este ejemplo, define una variable denominada `subset_fare`, que contendrá el conjunto de datos máximos y mínimos que se utilizarán para generar las barras deslizantes. Finalmente, la instrucción `st.write` ayudará a plasmar las barras deslizantes en la aplicación *web*.

Nota importante: Observa que se debe de anteponer la letra “f” al título que aparecerá debajo de la barra deslizante.

Paso 24. Con el *widget* de control deslizante (*slider*), especifica el nombre. Mientras tanto, establece los valores mínimos y máximos. Además del control deslizante, el código anterior incluye otra característica llamada expansor (`st.beta_expander`). Esta función es útil cuando deseas ocultar algunos *widgets* opcionales, y puedes especificar si deseas que el expansor se expanda de forma predeterminada.

Paso 25. Observa que la siguiente figura muestra un control deslizante interactivo de este tipo.

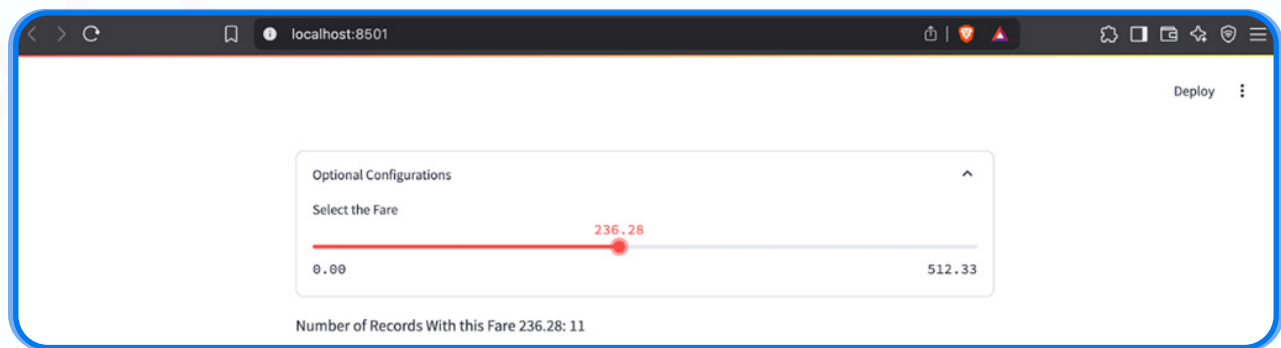


Figura 19. Expansor y control deslizable.

Paso 26. A continuación, observa el código completo y cómo ejecutarlo en **Visual Studio Code**. Para esto, crea el programa **widgetsApp.py**:

```
$ touch widgetsApp.py
```

Paso 27. Edita el **widgetsApp.py** con el código:

```
import pandas as pd

import streamlit as st

titanic_link = 'https://raw.githubusercontent.com/
mwaskom/seaborn-data/master/titanic.csv'

titanic_data = pd.read_csv(titanic_link)

# Create the title for the web app

st.title("My First Streamlit App")
```

```
sidebar = st.sidebar

sidebar.title("This is the sidebar.")

sidebar.write("You can add any elements to the
sidebar.")

# ask user the current date

today = datetime.date.today()

today_date = st.date_input('Current date', today)

st.success('Current date: `%s`' % (today_date))

# Display the content of the dataset if checkbox is
true

st.header("Dataset")

agree = st.checkbox("show DataSet Overview ?")

if agree:

    st.dataframe(titanic_data)

st.header("Data Description")
```

```
selected_class = st.radio("Select Class", titanic_
data['class'].unique())
```

```
st.write("Selected Class:", selected_class)
```

```
st.markdown("____")
```

```
selected_sex = st.selectbox("Select Sex", titanic_
data['sex'].unique())
```

```
st.write(f"Selected Option: {selected_sex!r}")
```

```
st.markdown("____")
```

```
optionals = st.beta_expander("Optional
Configurations", True)
```

```
fare_select = optionals.slider(
```

```
    "Select the Fare",
```

```
    min_value=float(titanic_data['fare'].min()),
```

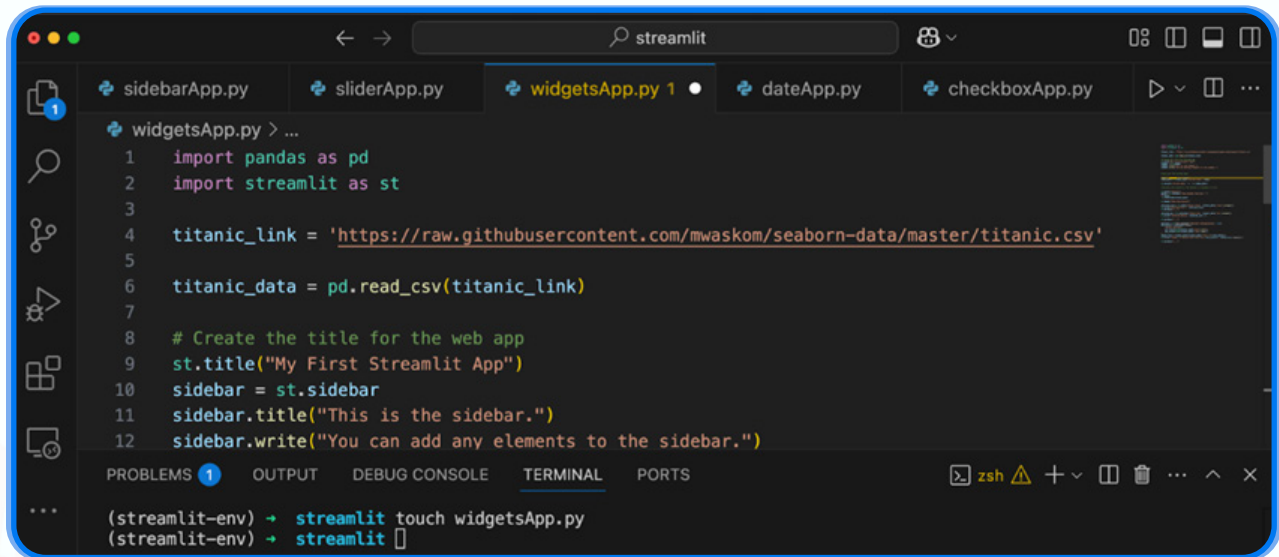
```
    max_value=float(titanic_data['fare'].max())
```

```
)
```

```
subset_fare = titanic_data[(titanic_data['fare'] >=
fare_select)]
```

```
st.write(f"Number of Records With this Fare {fare_
select}: {subset_fare.shape[0]}")
```

```
st.markdown("____")
```



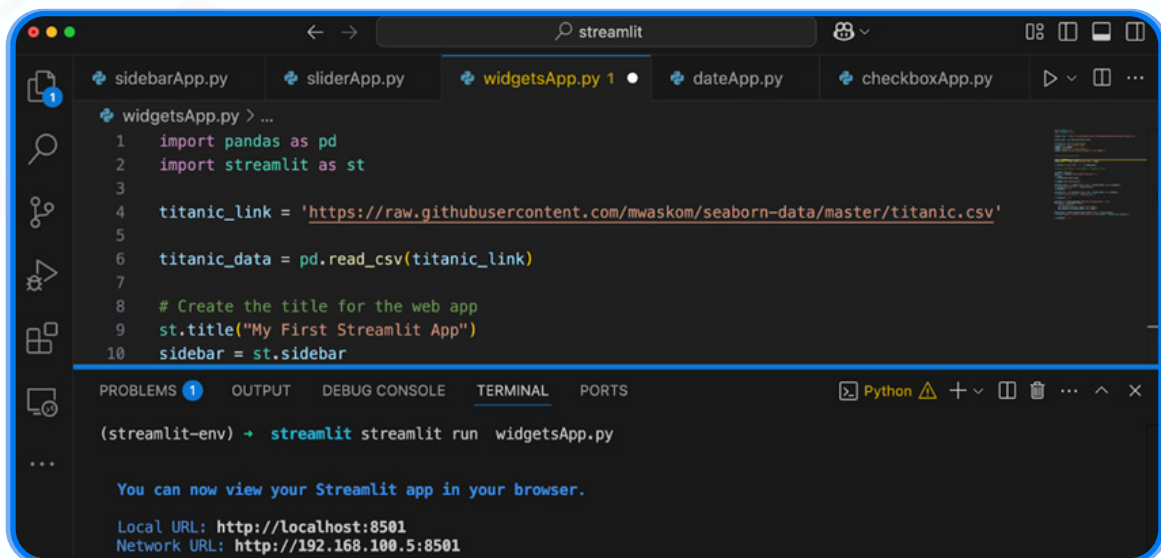
```
widgetsApp.py > ...
1  import pandas as pd
2  import streamlit as st
3
4  titanic_link = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv'
5
6  titanic_data = pd.read_csv(titanic_link)
7
8  # Create the title for the web app
9  st.title("My First Streamlit App")
10 sidebar = st.sidebar
11 sidebar.title("This is the sidebar.")
12 sidebar.write("You can add any elements to the sidebar.")

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
(streamlit-env) + streamlit touch widgetsApp.py
(streamlit-env) + streamlit
```

Figura 20. Edición de **widgetsApp.py**.

Paso 28. Ejecuta **widgetsApp.py** con:

```
$ streamlit run widgetsApp.py
```



```
widgetsApp.py > ...
1  import pandas as pd
2  import streamlit as st
3
4  titanic_link = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv'
5
6  titanic_data = pd.read_csv(titanic_link)
7
8  # Create the title for the web app
9  st.title("My First Streamlit App")
10 sidebar = st.sidebar

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
(streamlit-env) + streamlit run widgetsApp.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.100.5:8501
```

Figura 21. Edición de **widgetsApp.py**.

Paso 29. Al hacerlo, se cargará una pestaña en el navegador predeterminado. Visualiza la aplicación web que has creado, tal como se aprecia en la siguiente figura.

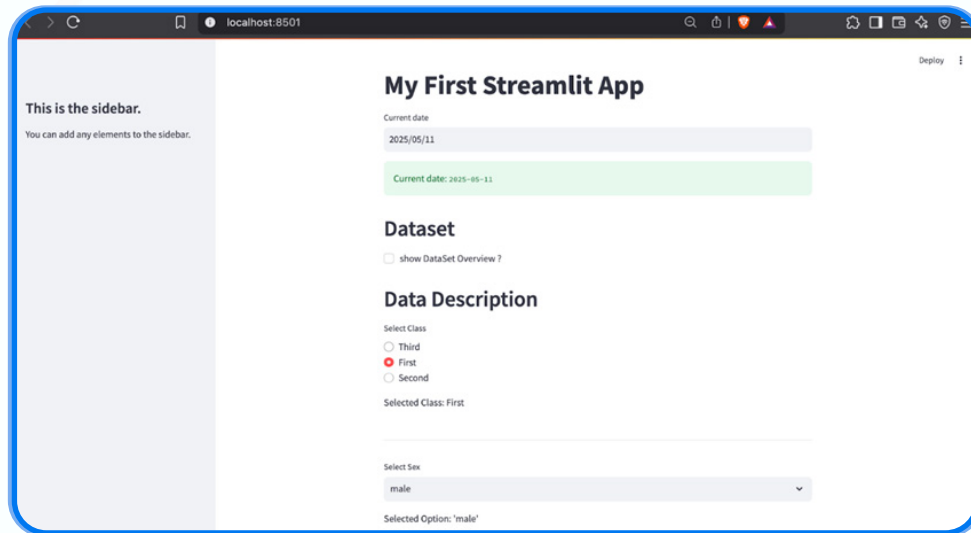


Figura 22. Vista de la aplicación web con **Streamlit** usando una barra lateral (*sidebar*) y los controles: botón de opción, casilla de verificación y control deslizante.

Hasta el momento, hemos revisado cómo generar los controles más comunes en **Streamlit** para una aplicación web, pero, cómo podrías construir un sitio web que contenga características específicas, por ejemplo:

- El código para generar un control de *checkbox* para las categorías: 1, 2 y 3.
- El código para generar un control de *selectbox* para las regiones: Norte y Sur.
- El código para generar un control de *slider* para máximo precio: 10.00, y mínimo precio: 5.00.

Te invitamos a revisar el siguiente enlace en donde encontrarás las instrucciones necesarias para hacerlo: https://docs.streamlit.io/en/stable/getting_started.html

Finalmente, es normal que en la actualidad las empresas utilizan **Streamlit** porque permite crear *dashboards* de manera muy práctica y sencilla sin la necesidad de ser experto en lenguajes de programación propios de desarrollo web. Por eso, los conocimientos de la infraestructura, los recursos de *hardware* y *software* requeridos para construir y ejecutar una aplicación en **Streamlit** son de vital importancia para desarrollar *dashboards* sofisticados.

Espacio para reflexionar

Con base en tus actividades profesionales, y en la información que manipulas todos los días, reflexiona sobre los siguientes aspectos:

- ¿Qué datos te sirven para adaptarlos a un cuadro de mandos basado en **Streamlit**?
- Si tuvieras que elegir tu propia base de datos, ¿qué tipo de información sería?, ¿qué estructura tendría tu *dashboard*?
- Con base en tu experiencia, ¿es más conveniente usar **Streamlit** en lugar de **Jupyter Notebook**?, ¿por qué?
- ¿Para qué tipo de usuarios consideras que es recomendable utilizar **Streamlit**?
- ¿Qué ventajas tiene para tus actividades diarias el utilizar una herramienta como **Streamlit**?

Recuerda que para construir una aplicación web que aloje los análisis que realizaste de los datos, es importante considerar y entender los datos a usar, determinar el modelo a utilizar para mostrar evidencias o hallazgos, pero cómo un científico de datos, lo más importante es considerar cómo vas a presentar todo el proyecto de una forma que sea fácil, amigable y sencilla de entender por el usuario final.

Lectura de *datasets* en formato CSV, JSON y de APIs externas

Las plataformas web regularmente consumen grandes cantidades de información de manera rápida e interactiva. Con **Streamlit** puedes consultar fuentes de información, usando otros paquetes como **Pandas** y **NumPy**.



Figura 1. Plataformas web.

Es momento de crear una aplicación simple en **Streamlit** que permita leer una base de datos en formato **.csv** o **.json**, y la despliegue como un *dataframe*. Pero ¿por qué es importante procesar los datos en formato **.csv**? Para que tengas una idea de la importancia de tratar los datos, debes saber que la gran mayoría de sitios web comparten información en este formato, así como sistemas de grandes corporaciones que usan productos como **SAP**. Para realizar esta aplicación, sigue estos pasos:

Paso 1. Importa la biblioteca **Streamlit**.

Paso 2. Importa la biblioteca de **Pandas**.

Paso 3. Lee el archivo usando `read_csv()` de **Pandas**.

Paso 4. Despliega usando la función `st.dataframe()` de **Streamlit**.

Paso 5. Posteriormente, descarga el **dataset** en el apartado archivos adjuntos de la plataforma.

Paso 6. En **Visual Studio Code**, asegúrate de que esté activo el ambiente virtual:

```
$ source streamlit-env/bin/activate
```

```
$ touch pandasApp.py
```

Paso 7. Edita **pandasApp.py** con el siguiente código:

```
import pandas as pd

import streamlit as st

names_link = 'dataset.csv'

names_data = pd.read_csv(names_link)

# Create the title for the web app
```

```
st.title("Streamlit and pandas")
```

```
st.dataframe(names_data)
```

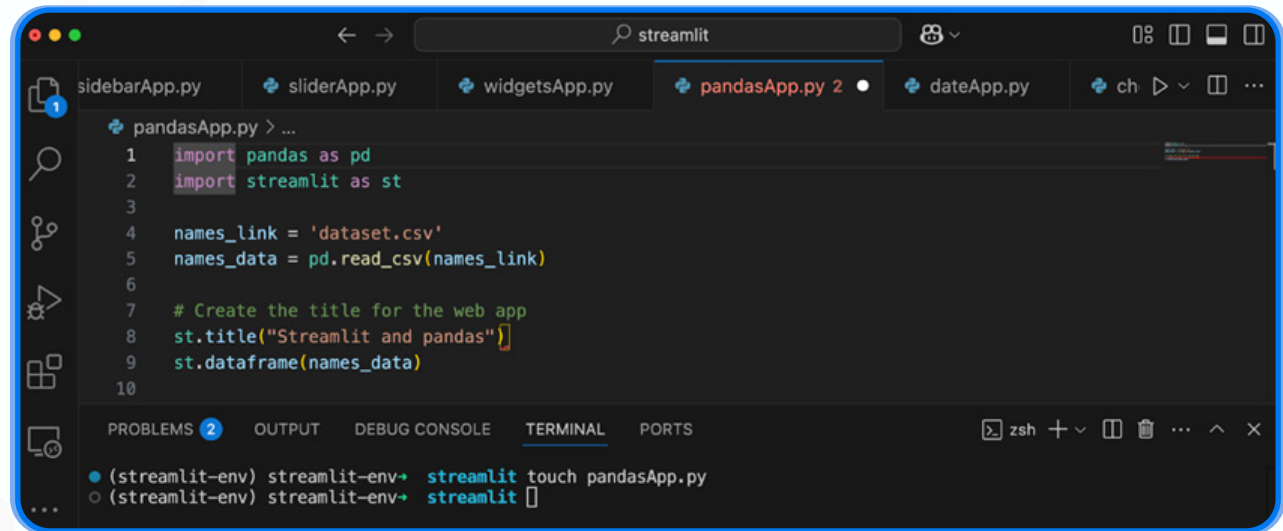


Figura 2. Edición de **pandasApp.py**.

Paso 8. Una vez creado y codificado, ejecuta el archivo con el comando **Streamlit** para que se visualice su contenido:

```
$ streamlit run pandasApp.py
```

Paso 9. Al hacerlo, la aplicación se abrirá en una sesión del navegador:

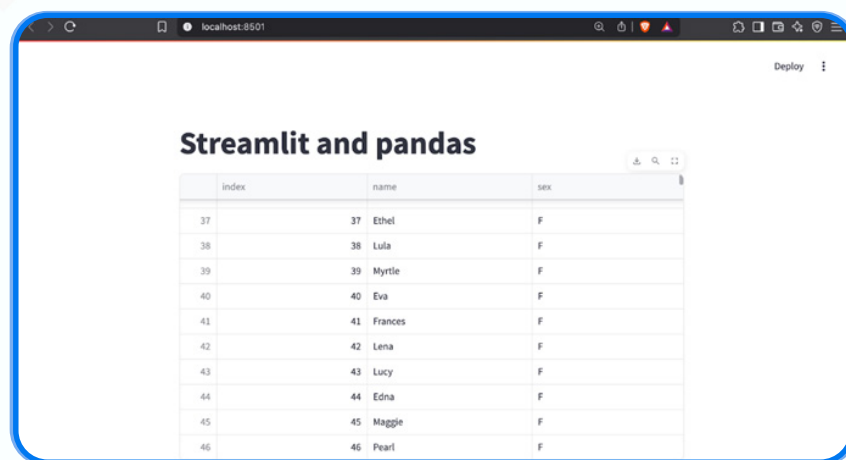


Figura 3. Aplicación en el navegador.

Continúa para conocer más sobre el estatuto *caching*.

Estatuto *caching*

Cuando lees datos desde una aplicación web, este proceso suele ser lento y consume muchos recursos de ancho de banda dependiendo de la cantidad de datos y la velocidad de internet. **Streamlit** tiene una propiedad que permite guardar en una memoria caché los datos leídos de una fuente de datos. Por lo tanto, si realizas una llamada idéntica, los datos son tomados del caché, de este modo, se evita el tráfico y/o consumo de datos nuevamente.

En ese sentido, **Streamlit** solo realizará llamadas nuevas a la fuente de datos si se cumplen algunas de las siguientes condiciones:

- El código fuente de la función que realiza la llamada ha cambiado.
- Ha cambiado la fuente de datos.
- Se están realizando llamadas con parámetros diferentes.

Para definir una función de **Python** como `cacheable`, puedes utilizar el atributo `@st.cache`. Para hacerlo, realiza lo siguiente:

Paso 1. En **Visual Studio Code**, crea:

```
$ touch cacheApp.py
```

Paso 2. Edita **cacheApp.py** con el código:

```
import pandas as pd

import streamlit as st

DATA_URL = 'dataset.csv'

@st.cache

def load_data(nrows):

    data = pd.read_csv(DATA_URL, nrows=nrows)

    return data

# Create the title for the web app

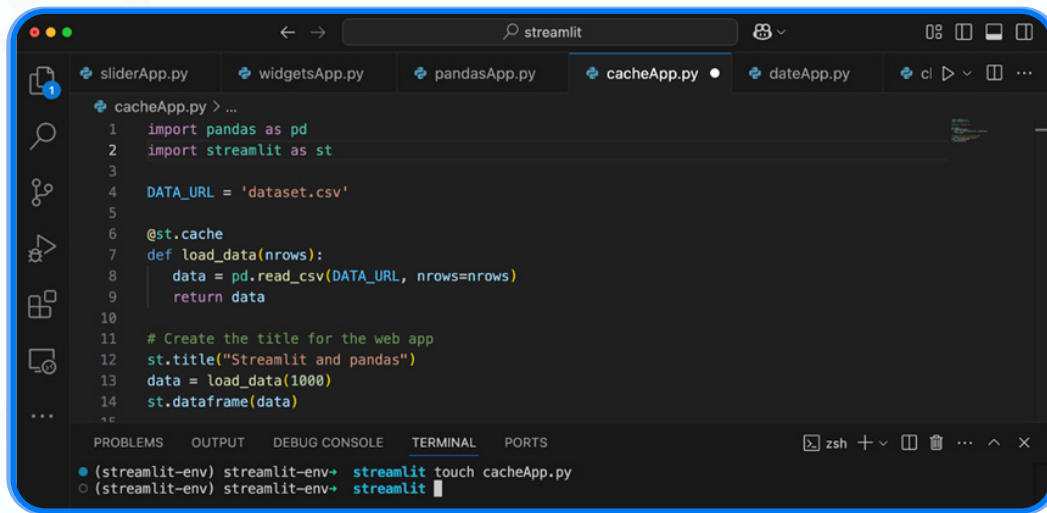
st.title("Streamlit and pandas")

data = load_data(1000)

st.dataframe(data)
```

Paso 3. Analiza que, en este ejemplo, creaste un *dashboard* en **Streamlit** que lee la fuente de datos del ejemplo anterior. Después, creaste una función `load_data()` que permite recibir como parámetro el número de registros a obtener, agregar el atributo `@st.cache` a la

función `load_data()`, finalmente, colocaste un título y mensajes para indicar cuando se haya iniciado y finalizado la carga de datos.



```
1 import pandas as pd
2 import streamlit as st
3
4 DATA_URL = 'dataset.csv'
5
6 @st.cache
7 def load_data(nrows):
8     data = pd.read_csv(DATA_URL, nrows=nrows)
9     return data
10
11 # Create the title for the web app
12 st.title("Streamlit and pandas")
13 data = load_data(1000)
14 st.dataframe(data)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

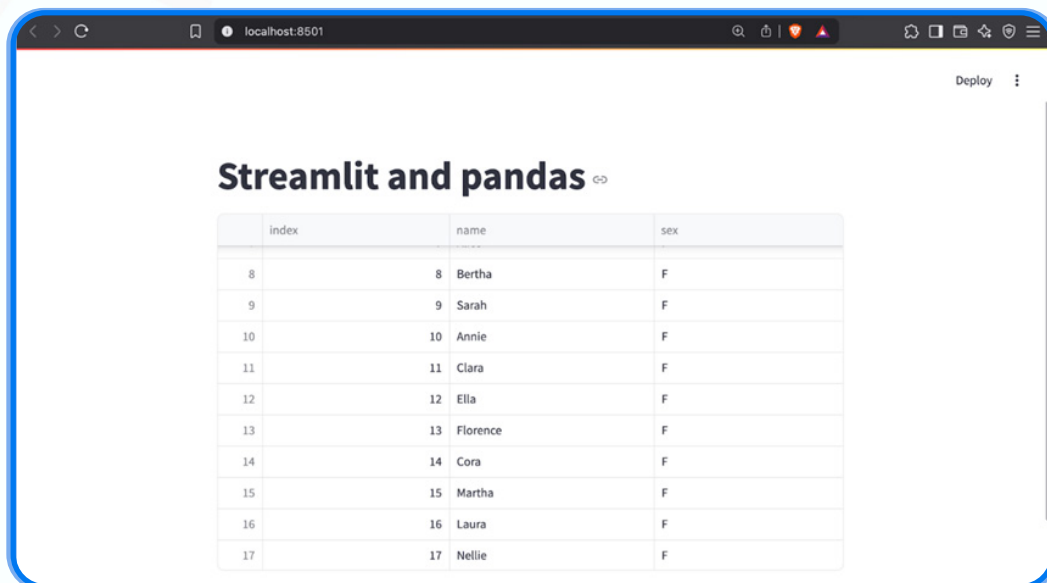
(streamlit-env) streamlit-env+ streamlit touch cacheApp.py
(streamlit-env) streamlit-env+ streamlit

Figura 4. Edición de **cacheApp.py**.

Paso 4. Ejecuta la aplicación:

```
$ streamlit run cacheApp.py
```

Paso 5. Abre la aplicación con el **URL** generado:



	index	name	sex
8	8	Bertha	F
9	9	Sarah	F
10	10	Annie	F
11	11	Clara	F
12	12	Ella	F
13	13	Florence	F
14	14	Cora	F
15	15	Martha	F
16	16	Laura	F
17	17	Nellie	F

Figura 5. Ejecución de la aplicación.

Nota: La aplicación tardará en la primera carga, si refrescas el navegador los datos se tomarán del caché porque se conservan las mismas propiedades de la llamada, entonces, la carga será mucho más rápida.

Interacción de datos con componentes básicos

La creación de *dashboards* interactivos con la plataforma **Streamlit** y **Python**, es posible usando componentes básicos como:

- **Textbox.**
- **Command Button.**
- **Selected box.**
- **SideBar.**
- **Checkbox.**

Control `text_input()`

Con **Streamlit** puedes incluir controles de texto para permitir al usuario capturar datos de entrada. Por una parte, con `st.text_input()` puedes definir un objeto para la entrada de datos, por ejemplo:

```
mynombre = st.text_input(' teclea tu nombre ')
```

Mientras que, con `st.text_input()` puedes validar y responder a la entrada de datos con una sentencia condicional `if`:

```
myname = st.text_input('nombre :')

if (myname):

    st.write("tu nombre es : {myname}")
```

También puedes crear una función y llamarla pasando como parámetro el dato capturado:

```
def bienvenida(nombre):

    mymensaje = 'bienvenido/a :' + nombre

    return mymensaje

myname = st.text_input('nombre :')

if (myname):

    mensaje = bienvenida(myname)

    st.write(" : {mensaje}")
```

Ahora, es momento de utilizar el mismo proyecto en **Visual Studio Code** para:

- Generar una nueva aplicación con **Streamlit** que se conecte a la base de datos **csv**.
- Crear un control `text_input()` para capturar un nombre o las primeras letras de un nombre, ejemplo: Mary, Mar, Jimmy, Jim.
- Visualizar en un *dataframe* las coincidencias encontradas del patrón de búsqueda.
- Imprimir el número de coincidencias.

Para posteriormente, crear tu tercera aplicación de **Streamlit** que demuestre el uso de `text_input`. Para lograrlo, sigue estos pasos:

Paso 1. En **Visual Studio Code**, crea **searchNames.py**:

```
$ touch searchNames.py
```

```
import streamlit as st
```

```
import pandas as pd
```

```
st.title('Streamlit - Search names')
```

```
DATA_URL = 'dataset.csv'
```

```
@st.cache_data
```

```
def load_data_byname(name):
```

```
    data = pd.read_csv(DATA_URL)
```

```
    filtered_data_byname = data[data['name'].str.  
contains(name)]
```

```
    return filtered_data_byname
```

```
myname = st.text_input('Name :')
```

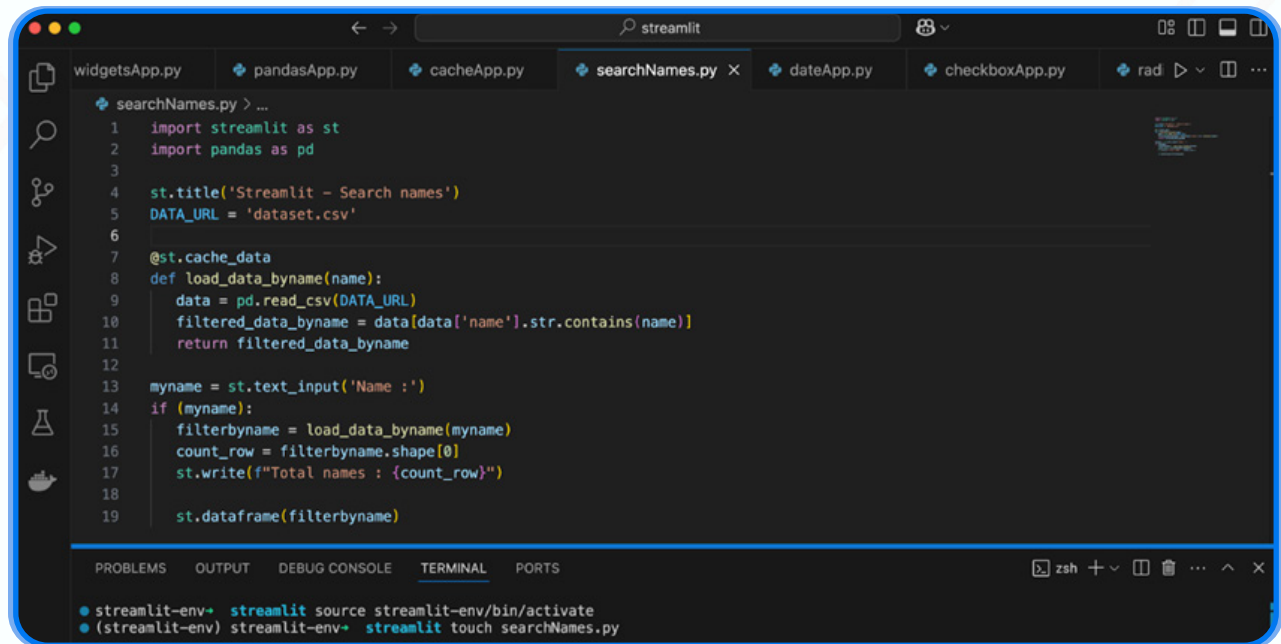
```
if (myname):
```

```
    filterbyname = load_data_byname(myname)
```

```
    count_row = filterbyname.shape[0]
```

```
    st.write(f"Total names : {count_row}")
```

```
    st.dataframe(filterbyname)
```



```
1 import streamlit as st
2 import pandas as pd
3
4 st.title('Streamlit - Search names')
5 DATA_URL = 'dataset.csv'
6
7 @st.cache_data
8 def load_data_byname(name):
9     data = pd.read_csv(DATA_URL)
10     filtered_data_byname = data[data['name'].str.contains(name)]
11     return filtered_data_byname
12
13 myname = st.text_input('Name :')
14 if myname:
15     filterbyname = load_data_byname(myname)
16     count_row = filterbyname.shape[0]
17     st.write(f"Total names : {count_row}")
18
19 st.dataframe(filterbyname)
```

streamlit-env+ streamlit source streamlit-env/bin/activate
streamlit-env+ streamlit touch searchNames.py

Figura 6. Edición de **searchNames.py**.

Paso 2. Ejecuta la aplicación:

```
$ streamlit run searchNames.py
```

Paso 3. Abre la aplicación con el **URL**:

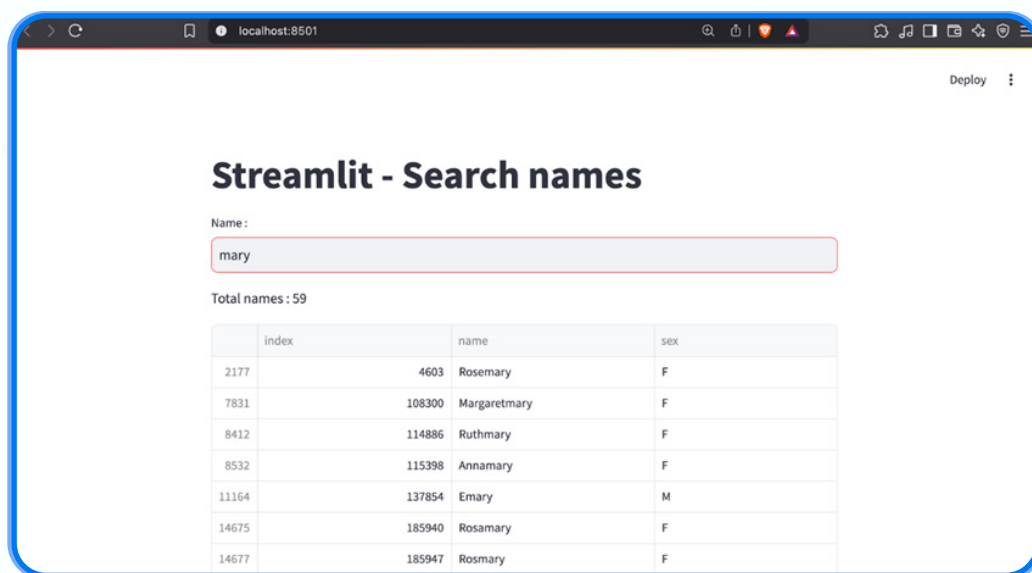


Figura 7. Ejecución de la aplicación.

Control `button()`

Streamlit ofrece la posibilidad de usar los controles tipo **Command Button** para que el usuario dispare una acción al dar clic sobre un botón. En ese sentido, con `st.button()`, puedes crear un botón de comando que responderá al clic de usuario. Regularmente, `st.button()` se asocia a una sentencia condicional `if` para detectar si el usuario ha hecho clic, como se muestra a continuación:

```
if st.button('Search') :  
  
    st.write("Has presionado el boton Search")
```

Es posible combinar `button` con otros controles como `text_input` para otorgar más funcionalidad, por ejemplo:

```
myname = st.text_input('name :')  
  
if st.button('Search') :  
  
    st.write(f"search name : {myname}")
```

Esto permite capturar un nombre en `myname`, crear un `button`, y responder al evento clic e imprimir el texto capturado. Ahora, es momento de utilizar el mismo proyecto de **Visual Studio Code** para:

- Generar una nueva aplicación **Streamlit**, que se conecte a la base de datos **csv**.
- Permitir buscar nombres de la base de datos por el campo `index`.

- Definir 2 `text_input`, un `button` y buscar un conjunto de nombres por el rango capturado.

Para crear una aplicación de **Streamlit** que demuestre el uso de `Button`, sigue estos pasos:

Paso 1. En **Visual Studio Code**, crea **range.py**, y edítalo con el código:

```
import streamlit as st

import pandas as pd

st.title('Streamlit - Search ranges')

DATA_URL = 'dataset.csv'

@st.cache_data

def load_data_byrange(startid, endid):

    data = pd.read_csv(DATA_URL)

    filtered_data_byrange = data[
        (data['index']>=startid) & (data['index']<=endid) ]

    return filtered_data_byrange
```

```

startid = st.text_input('Start index :')

endid    = st.text_input('End index :')

btnRange = st.button('Search by range')

if (btnRange):

    filterbyrange = load_data_byrange( int(startid),
int(endid) )

    count_row = filterbyrange.shape[0]

    st.write(f"Total items : {count_row}")

    st.dataframe(filterbyrange)

```

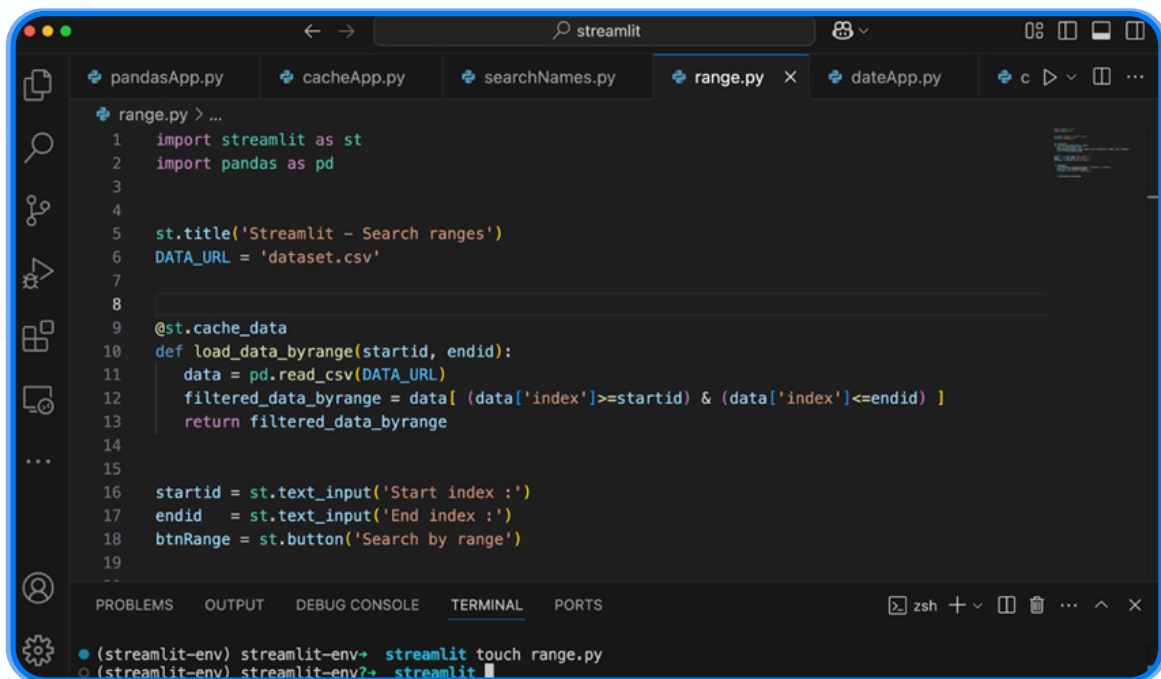


Figura 8. Edición de **range.py**.

Paso 2. Ejecuta la aplicación con:

```
$ streamlit run range.py
```

Paso 3. Observa cómo luce al abrirla en la aplicación con el **URL** generado:

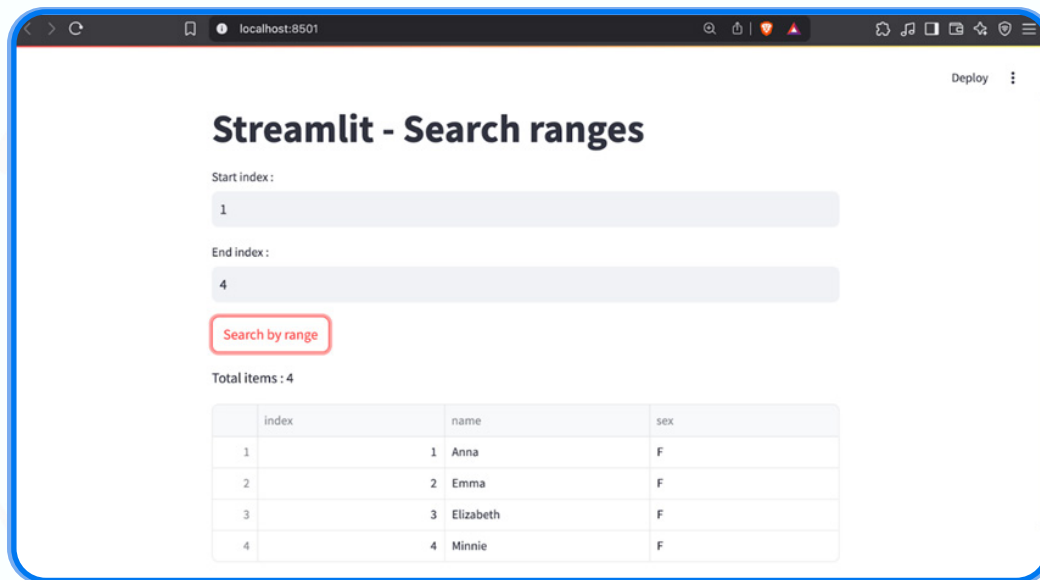


Figura 9. Ejecución de la aplicación.

Control Selectbox

El control `selectbox` permite crear listas desplegables a partir de un conjunto de datos. La fuente de datos puede ser una lista, una columna de *dataframe*, entre otros. Con `st.selectbox()` puedes crear una lista desplegable, indicando el mensaje a mostrar, y la fuente de datos, como se muestra a continuación:

```
selected_sex = st.selectbox("Select Sex",  
data['sex'].unique())
```

Para este ejemplo, `data` es un *dataframe* con una columna llamada `sex`.

Aquí puedes usar el dato seleccionado para filtrar datos de un *dataframe*:

```
selected_sex = st.selectbox("Select Sex",
                             data['sex'].unique())

st.write(f"Selected Option: {selected_sex!r}")

filtered_data_sex = data[data['sex'] == selected_sex]
```

Al hacerlo, creará una lista desplegable con la columna del *dataframe*, guardará en `selected_sex` el valor seleccionado, y creará un nuevo *dataframe* con los registros que sean iguales al valor seleccionado.

Posteriormente, utiliza el mismo proyecto de **Visual Studio Code** para:

- Generar una nueva aplicación **Streamlit** que se conecte a la base de datos **csv**.
- Definir un `selectbox` y un `button`.
- Asocia el control `selectbox` al campo `sex` del *dataframe*.
- Establecer que, al dar clic en el botón, filtre por el valor seleccionado indicado la cantidad de hombres o mujeres de la base de datos.

Continúa creando tu aplicación de **Streamlit** para demostrar el uso de `selectbox`. Para esto, sigue estos pasos:

Paso 1. En **Visual Studio Code**, crea **filterBySex.py**:

```
import streamlit as st

import pandas as pd

st.title('Streamlit - Filter by sex')

DATA_URL = 'dataset.csv'

@st.cache_data

def load_data():

    data = pd.read_csv(DATA_URL)

    return data

@st.cache

def load_data_bysex(sex):

    data = pd.read_csv(DATA_URL)

    filtered_data_bysex = data[ data['sex'] == sex ]

    return filtered_data_bysex
```

```

data = load_data()

selected_sex = st.selectbox('Select sex',
data['sex'].unique())

btnFilterbySex = st.button('Filter by sex')

if (btnFilterbySex):

    filterbysex = load_data_bysex( selected_sex )

    count_row = filterbysex.shape[0]

    st.write(f"Total items : {count_row}")

    st.dataframe(filterbysex)

```

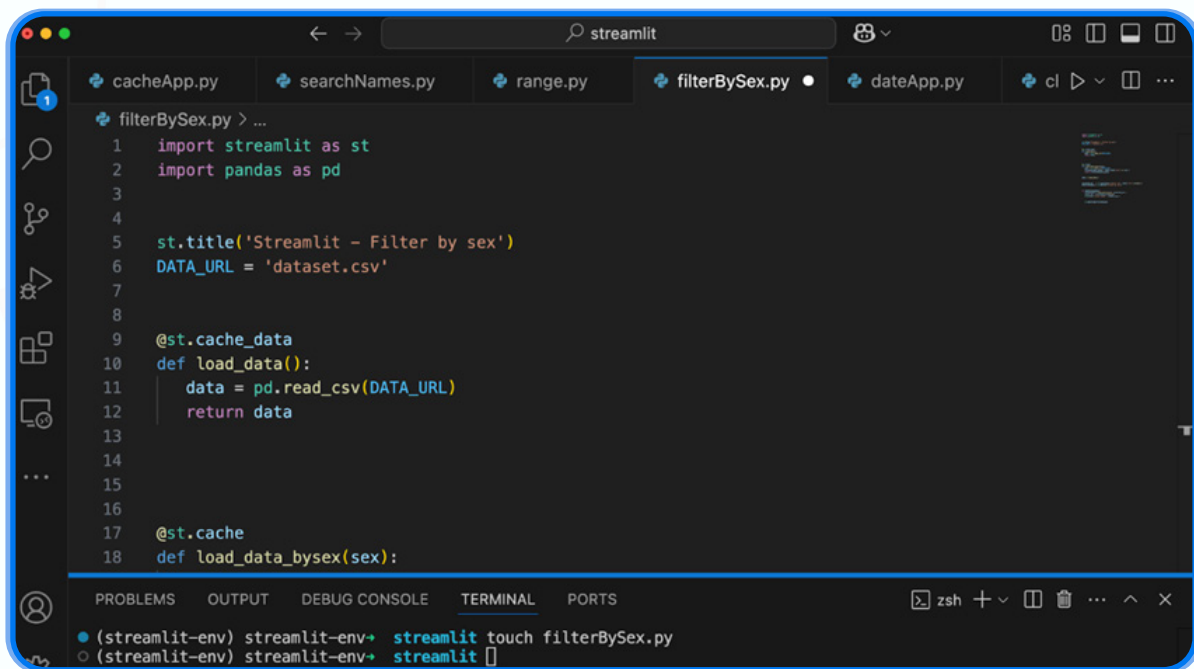


Figura 10. Edición de **filterBySex.py**.

Paso 2. Ejecuta la aplicación con:

```
$ streamlit run filterBySex.py
```

Paso 3. Observa cómo luce al abrirla con el **URL** generado:

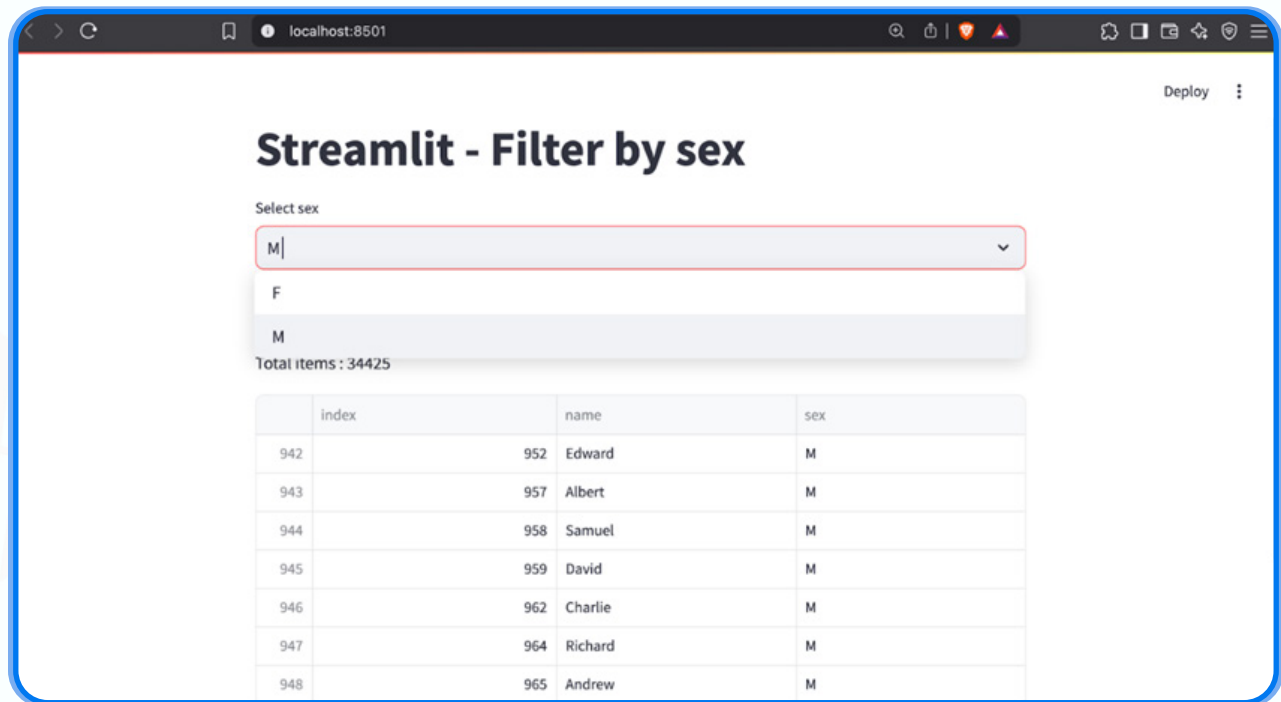


Figura 11. Ejecución de la aplicación.

Streamlit usa **Pandas** para la visualización de datos, además, permite manejar la memoria caché entre llamadas a la fuente de datos para evitar tráfico innecesario. En esencia, es una potente herramienta para crear aplicaciones web con acceso a datos.

Espacio para reflexionar

Pandas y **Streamlit** trabajan de forma transparente para acceder a los datos y realizan el proceso de caché de forma automática con `st.cache()`, ¿en tu organización a qué aplicaciones le darías al acceso de datos con caché para minimizar tiempos de carga y reducir el tráfico de datos?

Por su parte, el uso de *widgets* como el control `input_text` permite crear aplicaciones web interactivas, ¿a qué aplicaciones le darías `control input_text`?, ¿cuántos son necesarios?, ¿requerirías controles adicionales para tener más funcionalidad?

En esencia, **Streamlit** ofrece muchos controles de usuario interactivos útiles para crear *dashboards* con acceso a datos de una manera muy amigable, pero a la vez potente.

Pruébate

Ejercicio de reflexión

Hasta el momento has estudiado los primeros pasos para trabajar con la plataforma **Streamlit**, al respecto, reflexiona los siguientes aspectos:

- Con base en tus actividades profesionales, ¿qué ambiente utilizarías para instalar **Streamlit**?, ¿Windows o Mac?
- Con base en tu experiencia, ¿cuáles son las limitaciones y ventajas de las metodologías para el desarrollo de proyectos de ciencia de datos?
- Si un cliente te solicita una sugerencia para empezar un proyecto de ciencia de datos, ¿qué metodología recomendarías?

Cuestionario

Instrucciones: Lee con atención cada pregunta y selecciona una respuesta.

1. Un científico de datos no exige requisitos técnicos para construir dashboards en Streamlit. El enunciado es ...

a. Falso

b. Verdadero

Retroalimentación: El enunciado es verdadero. **Streamlit** no requiere habilidades especiales en programación web para construir *dashboards*.

2. ¿Cuál es la función de Streamlit que permite colocar el título de un *dashboard*?

a. `st.write()`

b. `st.title()`

c. `st.header()`

Retroalimentación: **Streamlit** utiliza la función `st.title()` para visualizar el título de una aplicación.

3. ¿Cuál es el módulo de **Python** que usa **Streamlit** para manipular datos de un **CSV**?

a. TensorFlow

b. NumPy

c. Pandas

Retroalimentación: **Pandas** es utilizado para conectarse a una fuente de datos **CSV** y manipularlos como *dataframes*.

4. ¿Cuál es la función en **Streamlit** que permite visualizar un *dataframe* en **Pandas**?

a. `st.dataframe()`

b. `st.write()`

c. `read_csv()`

Retroalimentación: Es posible leer un **CSV** con **Pandas** y visualizarlo como un *dataframe* usando `st.dataframe()`.

5. ¿Cuál es el atributo de **Streamlit** que permite almacenar datos de forma temporal para evitar llamadas idénticas a una fuente de datos?

a. `st.text_input()`

b. `@st.cache`

c. `def myfuncion()`

Retroalimentación: Con el atributo `@st.cache` es posible etiquetar una función para que sea `cacheable`.

Ideas para llevar

- Identificaste que cuando se está construyendo una plataforma para el análisis de datos a través de cuadros de mando, es importante considerar aspectos básicos cómo:
 - Los usuarios que utilizarán esta herramienta.
 - El contenido que estará en el análisis.
 - La información más importante para mostrar.
 - El gráfico principal para que el usuario comience su análisis.

- Identificaste las características de los ambientes de trabajo Windows y Mac en el desarrollo de proyectos.
- Examinaste que para iniciar a usar la plataforma **Streamlit** en la construcción de aplicaciones web para proyectos de analítica, es importante considerar la información o datos por analizar, los que van a interactuar con el análisis, así como los hallazgos o evidencias por mostrar.
- Interiorizaste que para comenzar a trabajar en **Streamlit** es necesario determinar en el ambiente: Windows o Mac.
- Analizaste que los controles de texto, botones de comando y `selectbox` son sumamente útiles en todos los formularios de captura de datos para realizar la acción interactiva con los usuarios de los *dashboards*.
- Interiorizaste que **Streamlit** ofrece muchos controles de usuario interactivos útiles para crear *dashboards* con acceso a datos de una manera muy amigable, pero a la vez potente.

Bibliografía

Anaconda. (2021). *Installing Anaconda Documentation* [Instalación de la documentación de Anaconda]. Anaconda Inc. Recuperado el 13 de mayo de 2025 de: <https://docs.anaconda.com/anaconda/install/windows/>

Streamlit. (2021). *Install Streamlit* [Instalar Streamlit]. Streamlit Inc. Recuperado el 13 de mayo de 2025 de: <https://docs.streamlit.io/en/stable/troubleshooting/clean-install.html>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 13 de mayo de 2025 y su disponibilidad depende del autor.

Material adicional

Streamlit. (2025). *Get started with Streamlit* [Empezar a utilizar Streamlit]. Streamlit. Recuperado el 06 de junio de 2025 de: https://docs.streamlit.io/en/stable/getting_started.html

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 13 de mayo de 2025 y su disponibilidad depende del autor.